

Orbital Stellar

Reframing the Thesis, Architecture & Monetization Strategy — a working addendum to the April 2026 Strategic Intelligence Report

PREPARED

April 22, 2026

CLASSIFICATION

Internal — Strategic Planning

SCOPE

Thesis, Architecture, Monetization

SUPERSEDES

N/A — Supplements April 14 report

HORIZON

20 Years (2026–2046)

OWNER

Founding Team / CEO

Executive Summary

The April 14 Strategic Intelligence Report is a well-researched 20-year plan for Orbital Stellar. This addendum does not contradict it — it *upgrades* it on three axes where the report quietly self-limits:

1. **Framing.** The report calls Orbital "event infrastructure" for 20 pages but by Era III is actually building an identity layer, a payment gateway, a compliance cloud, and a data market. Those are runtime products, not webhook products. Reframing Orbital as *Stellar's programmable runtime* moves the TAM from ~\$500M to \$50B+ without a single line of additional engineering.
2. **Primary risk.** Engineering is not the risk — Stellar's dev-ecosystem density is ~5% of Ethereum's. Indexed to "Stellar developers," the ceiling is ~\$6M ARR and the business stalls in Era II. Indexed to "regulated money movement that happens to run on Stellar" (remittances, anchors, CBDCs, agent payments), the TAM is 100× larger and the competitive set collapses.
3. **Monetization architecture.** The report's open-core thesis is correct but under-specified. The right structure is one company, one brand, three tiers: *Orbital Core* (MIT, free) → *Orbital Cloud* (hosted developer SaaS) → *Orbital Rails* (regulated-money enterprise). Each tier is a strict superset of the one below, with moat increasing monotonically through operations, certifications, and cryptographic trust.

One strategic call to make this quarter: decide whether Orbital's north star is a *developer tool* or a *regulated-money infrastructure layer*. The current code serves both, but the GTM, pricing, hiring, and pitch diverge hard after Series A. The recommendation in this document is the latter — it is where Stellar's technical asymmetry is, where the founding team's Harvard research + serial-CEO profile compounds, and where \$200M+ ARR is actually reachable.

Where the Existing Report Quietly Self-Limits

The framing problem

Framing is not decoration. Framing is a *filter* that decides which opportunities appear as "obvious" versus "scope creep" to your team, investors, and customers. The current framing — "event infrastructure for Stellar" — is smaller than the thesis the report actually executes on.

Evidence the framing is too small

Read Era I–IV in sequence. In Era I you ship webhooks. By Era III you ship:

- **An agent payment SDK** (x402 middleware, agent-sdk, payment-gateway)
- **An identity standard** (passkeys, embedded wallets, federated auth)
- **A compliance infrastructure layer** (FATF audit logs, anchor lifecycle events)
- **A data market** (archival API, institutional licensing)
- **A decentralized operator network** (Orbital Network)
- **A CBDC partnership business**

A reasonable observer would not call this "event infrastructure." They would call it a *programmable runtime + payments platform + identity layer + compliance cloud*. Four separate product categories, stacked.

Why the framing matters (three audiences)

1. **Your own team.** An engineer in Year 3 asks "why are we building an identity SDK? We're an events company." If the answer requires a 10-minute explanation, the framing is wrong.
2. **Investors.** A Series B VC reading a deck titled "Events Infrastructure for Stellar" price-anchors to Svix (\$40M Series A), not Stripe (\$95B). The framing itself caps the valuation.
3. **Enterprise buyers.** A CBDC architect does not procure "webhook infrastructure" at the board level. She procures "programmable money rails." The word on the first slide determines which budget line you draw from.

The TAM numbers

Framing	Comparable Companies	Category Ceiling
"Webhook infrastructure for Stellar developers"	Svix (~\$10M ARR), Hookdeck (~\$15M ARR), Sorobanhooks (negligible)	~\$300–500M globally
"Developer platform + payment rails + compliance cloud for Stellar-adjacent finance"	Stripe core (\$1B+ ARR), Twilio (\$4B+), Plaid (\$1B+), Chainalysis (\$150M+), Fireblocks (\$300M+)	\$50B+ globally

Same team. Same code. Different frame. **100× TAM expansion with zero additional engineering.**

Seven Moves the Report Does Not Contain

Any one of these is a 10–100× unlock beyond the current 20-year plan. Each is feasible with 2026-era engineering; none require unsolved research.

1. Intent settlement, not event notification

x402 ("agent pays for API call") is table-stakes. The real frontier is *intent resolution*: agents/users post "swap 100 USDC→XLM best price in next 60s, retry via anchors if DEX fails" and Orbital's runtime atomically resolves across DEX + path-payments + anchors + external rails. CoW Swap and UniswapX are doing this for EVM. **Zero competitors on Stellar.** The event engine already in the repo is 60% of an intent-matching engine — the team has built it and not yet realized it.

2. Event markets as a priced primitive

Once Orbital owns the deepest Stellar archive (report's Era III), stop thinking "data API pricing." Sell *structured access*: first-look windows, latency tiers, exclusivity per-contract. Market makers, MEV searchers, compliance desks, anchor treasuries pay different prices for the same event. This is Bloomberg-terminal economics applied to on-chain events. Nobody has shipped it as a productized market — not even on EVM.

3. ZK-attested compliance gateway

The report treats compliance as a SOC 2 checkbox. The real play: every webhook carries a *zero-knowledge proof* that the underlying transaction satisfies FATF travel rule / sanctions / source-of-funds *without revealing inputs*. Regulated consumers get cryptographic trust, not vendor-trust. Collapses a 6-month enterprise sales cycle into a 2-week technical integration. 2026 ZK tooling (Noir, RiscZero) makes this feasible this year, not in Era III.

4. Stellar Shadow-Fork (simulator)

Tenderly's \$500M valuation is almost entirely "fork mainnet, replay, debug." Stellar has nothing equivalent. Fork any ledger, inject hypothetical ops, replay Soroban invocations, measure MEV/slippage/compliance impact. This eats four buyer-budgets at once: debugging, audit, simulation, pre-trade TCA. Built in Era II alongside the hooks — same data spine.

5. Parametric event-delivery insurance

The report says "reliability is the product." Monetize the claim: if Orbital misses a webhook, auto-pay a parametric payout from a collateralized pool. Converts reliability from marketing claim into balance-sheet product. Regulated buyers (banks, anchors) cannot legally consume

infra without this once it exists. Competitors cannot copy fast — they need years of SLA data to underwrite it.

6. Flip the ABI registry from moat to standard

The report's plan to keep the Soroban ABI Registry proprietary is 2015 thinking. Etherscan's registry won *because* it is effectively a public good. Donate the registry to an SEP-governed DAO; operate the verification/API layer. Lose a moat, win the category. Standards compound harder than IP in infrastructure.

7. Migrate Era IV from SaaS to oracle/accreditation model

By Year 10, pure-SaaS margin compresses. The durable position is rating, insuring, and certifying other Stellar infrastructure — "Moody's for anchors / contracts / relayers." A flywheel SaaS structurally cannot catch.

Feasibility Assessment

The tech is easy

Stellar's Horizon/RPC gives you a firehose; Soroban emits events; you run a stream processor (Kafka / NATS / Redpanda), a rule engine, and a delivery layer. A competent four-person team ships the MVP in a quarter. **None of it is unsolved research.**

The hard part is who pays

Stellar's dev ecosystem is ~5% of EVM's means: 5,000–8,000 active devs vs. Ethereum's 150,000+. If the entire buyer pool is "Stellar developers who want webhooks," the ceiling is:

5,000 devs × 5% conversion × \$49/mo = **~\$150K MRR ceiling.**

That is a lifestyle business, not a \$200M ARR company.

This is why QuickNode charges \$49/mo — they know the dev-tool ceiling is low and make it up on volume across 30 chains.

The repositioning flip

Same codebase, different buyer:

- MoneyGram processes ~\$150B/yr cross-border. They're on Stellar.
- Circle's USDC on Stellar handles billions in monthly volume.
- 60+ anchors in regulated corridors (Nigeria, Kenya, Philippines, Brazil).
- Franklin Templeton's BENJI money-market fund settles on Stellar.

These entities do not buy **\$49/mo webhooks**. They buy **compliance-grade observability contracts at \$50K–\$500K/year per deployment**. A single MoneyGram-tier contract replaces 1,000 indie dev subscriptions. And they cannot buy from QuickNode — QuickNode's generic EVM-first posture does not meet their regulatory or Stellar-specific needs.

Same engineering. 100× revenue per customer. Completely different sales motion. *Feasibility is a function of positioning, not technology.*

Beyond Current Research — The Three-Plane Framework

Most prior art treats monitoring as a *dashboard problem*. Reframe it as three things nobody has fused. Each is a standalone research frontier; fusing them is the 100,000× move.

Control plane — intent → graph compilation

Today: a developer configures `IF contract == X AND event == swap AND amount > 1000 THEN webhook Y`. DSL-based: rigid, non-composable.

Intent model: the developer writes a goal in natural language or high-level spec:

"Alert me when any anchor's USDC reserves diverge from their attested off-chain holdings by more than 2% for 30+ minutes."

A compiler (LLM + type-checker) translates this into a *subscription graph*: a DAG of streaming subscriptions (Horizon trustline changes + off-chain attestation feed + time-windowed aggregator + comparator node + alert sink). The graph is signed, versionable, and composable. Users fork, extend, subscribe to each other's graphs.

Settlement plane — reactors + priority-fanout on Soroban

Today: event fires → HTTP POST → your code does something.

Reactor model: event fires → a Soroban smart contract function executes *atomically with detection*. The reactor is a contract users deploy that defines a response: auto-rebalance a liquidity pool, auto-top-up a payment channel, auto-freeze a trustline on suspicious volume.

Priority-fanout market: when the same event is relevant to 100 subscribers, latency matters asymmetrically. A market-maker pays a premium to receive the event 50ms before retail subscribers. That premium is auctioned per-block. This is MEV-style value capture for event delivery, denominated in XLM, settled on Soroban.

Uniquely Stellar: EVM cannot do this cleanly — it lacks Stellar's native path-payments, DEX, and anchor primitives. Orbital's reactor can respond with an economically-atomic multi-hop payment in the same transaction. A primitive EVM tools cannot structurally replicate.

Trust plane — ZK attestations for regulated consumers

Today: Orbital sends a webhook saying "transaction T happened." The bank trusts Orbital. That trust is a procurement blocker — SOC 2, PCI-DSS, legal review, annual audits.

ZK model: every outbound webhook carries a succinct zero-knowledge proof that (a) transaction T was in ledger L, (b) ledger L is canonical Stellar state, (c) the rule R evaluated to true on T. The bank verifies the proof locally in milliseconds. It never has to trust Orbital — only the math.

The fusion

Nobody has built all three. Tenderly has a weak version of plane 2 (simulations, not reactors). Blocknative has a weak version of plane 1 (rules, not intents). Polyhedra / Succinct have plane 3 for generic proofs but not bolted to observability. Orbital's novelty is the *fusion*, and Stellar is the first chain where the fusion is economically coherent.

OSS-contribution hook: if the Stellar Foundation judges novelty, submit the reactor + priority-fanout spec as an *SEP draft*. That is the move that gets SDF attention. You are not using the ecosystem — you are extending it.

The Strategic Decision — Dev Tool vs. Regulated-Money Infrastructure

The same code serves both. Everything else diverges. The first 10 hires and the first 3 customers cement the trajectory — this decision is irreversible by Series A.

Dimension	"Dev Tool" Path	"Regulated-Money Infra" Path
First customer	Indie Stellar developer @ \$49/mo	Anchor / fintech @ \$5K–\$50K/mo
Pricing unit	Per-event, per-webhook	Per-corridor, per-rail, per-attestation
Sales motion	Self-serve, PLG, Discord, content	Enterprise, 3-month cycles, SOC 2 upfront
GTM metric	npm downloads, DAUs, Discord members	LOIs, regulatory approvals, corridor coverage
Hiring priority	DX engineers, devrel, content	Solutions engineers, compliance lead, BD
Funding thesis	"Stripe for Stellar events"	"SWIFT-with-receipts for crypto-fiat rails"
Funding ceiling	\$5–20M Series A	\$50–200M Series B/C plausible
Board / investors	Dev-tool VCs (Redpoint, Kleiner)	Fintech/infra VCs (Ribbit, Bain Capital)
Exit universe	Acquired by QuickNode/Alchemy \$50–300M	Stripe/Visa/Circle acquisition or IPO at \$2B+
Competitive set	QuickNode, Moralis, Sorobanhooks	Chainalysis, Fireblocks, TRM Labs
20-year ceiling	~\$200M ARR (report's number)	\$1B+ ARR plausible

Why regulated-money wins (three reasons specific to Orbital)

1. **Stellar's asymmetry.** Stellar was literally designed for cross-border regulated money movement. Its technical advantages (3-second finality, sub-cent fees, anchors, path

payments, built-in DEX) are wasted on indie NFT projects. They are *purpose-built* for regulated rails. You are fighting with your tailwind, not against it.

2. **Team asymmetry.** A CEO who has built 100 startups and Harvard-graduate researchers are the wrong profile to win a devrel-led PLG race against QuickNode's 250,000 existing developers. That is a knife fight you are structurally disadvantaged in. But they are *exactly* the right profile to sit across from a Deutsche Bank compliance director or a Nigerian Central Bank CBDC architect. Use the knife you have.
3. **Defensibility.** Dev tools get out-shipped. Regulated infrastructure gets grandfathered. A compliance integration with MoneyGram is worth 5 years of feature development — replacing it internally would take them 18 months and require regulator re-approval. This is why TRM Labs, Chainalysis, and Fireblocks all built enterprise-first and all became unicorns; Dune, Tenderly, and Alchemy built dev-first and most are stuck at \$50–300M valuations.

The dual-track concession: you do not have to pick publicly. In Year 1, Orbital ships the open-source SDK (low-cost, developer cred, grant funding) *while* the founding team spends every spare hour on enterprise pipeline. Open source generates the credibility; enterprise generates the revenue. After Series A, allocation flips to ~80% enterprise. The mistake is letting the open-source side define the company's identity for three years before discovering the revenue ceiling.

Open-Source Strategy — What to Open, What to Monetize

Open source is essential, not optional. The critique of the report's "proprietary ABI registry" is *not* "close more code." It is "**open-source more aggressively**, but be strategic about what you open-source (code, standards, schemas) versus what you monetize (operations, certifications, SLAs, derived data)."

What stays MIT-licensed forever

- SDKs: `@orbital/hooks` , `@orbital/payments` , `@orbital/auth` , `@orbital/x402` , `@orbital/agent-sdk`
- Event schemas and webhook specifications
- The intent DSL / graph compiler
- Reference implementations of reactor contracts
- Client libraries for every supported language
- The Soroban ABI Registry's *data and schema* (operated, not owned)

Open-sourcing these is not charity — it is the **single most effective customer acquisition strategy** in developer infrastructure. Every successful peer (Supabase, Vercel, Grafana, HashiCorp, MongoDB, Clerk, Neon, Auth0, Elastic) follows this exact model. Lock-in comes from ergonomics and operated infrastructure, not from closed code.

What Orbital monetizes (never open-source)

- Operated infrastructure: hosted nodes, archival database, delivery network, SLAs
- Compliance certifications and attestations: SOC 2, PCI-DSS, ZK-proof generation at scale
- Enterprise features: audit logs, role-based access, dedicated tenancy, regulator reporting
- Proprietary datasets derived from operations (aggregate analytics, not raw event data)
- Priority/latency tiers in the fanout market
- Insurance / underwriting on event delivery

Rule: open-source captures the category (standards, mindshare, acquisition). Operations captures the revenue (uptime, certifications, dedicated support). Never confuse the two.

Company Structure — One Brand, Three Tiers

Why not separate products / brands

- **Brand halo.** Every OSS install of `@orbital/hooks` is a free trust signal that closes enterprise deals. Different names = compliance officer Googling you does not find the 10K GitHub stars. You pay for trust twice.
- **Talent leverage.** Same engineer should write the MIT SDK and the enterprise compliance layer. Different codebases = duplicated effort, drift, two mediocre products.
- **Capital efficiency.** Two brands = two websites, docs sites, PR motions, hiring pipelines. Fatal overhead for a seed-stage company.
- **The precedent is unanimous.** Every successful open-core company runs one brand with tiered products: Supabase, Vercel, Grafana, HashiCorp, MongoDB, Clerk, Neon, Auth0, Elastic. **Zero** split into separate brands.

Corporate structure

Orbital, Inc. (Delaware C-corp)
├─ Orbital Foundation (optional, Year 3+) – neutral governance for specs / SEPs
└─ Commercial operations (everything else)

The Foundation is optional and comes later — a credibility move for when regulators / SDF start asking "who controls the standards?" You can run fine without it in Years 1–3.

Product tiers

Product	License	Audience	Pricing	Owner
Orbital Core	MIT	Everyone	Free forever	OSS team
Orbital Cloud	Commercial	Startups, indie devs	\$0 / \$29 / \$149 / \$499 per month	PLG team
Orbital Rails	Commercial + certified	Fintechs, anchors, banks, CBDCs	\$50K–\$500K / yr	Enterprise team

Brand architecture

One logo, one color system, one voice, three pages:

- `orbital.dev` — developer home. GitHub stars, SDK docs, "npm install @orbital/hooks" above the fold. Looks like Supabase or Vercel.
- `orbital.dev/cloud` — hosted tier. Pricing calculator, self-serve signup.
- `orbital.com/rails` — regulated-money home. Compliance certs, customer logos, case studies, "talk to sales." Looks like Stripe Enterprise or Chainalysis.

Engineering rollout

- **Year 1 (pre-Series A, ~4–8 engineers):** single team, no splits. Everyone works across Core, Cloud, and early Rails features. Critical: architect Core with clean extension points so enterprise features bolt on without forking.
- **Year 2–3 (post-Series A, ~15–30 engineers):** Platform team (5–8) owns Core + Cloud shared infra; DX team (3–5) owns SDKs / docs / OSS community; Enterprise team (5–10) owns Rails-specific features. Core engineers rotate every 12–18 months so OSS stays first-class.
- **Year 4+ (~50+ engineers):** teams align to product lines but Platform still owns the shared spine.

Timing

Stage	Decision
Months 0–6	Single product, single team. Build Core + early Cloud. Internal doc notes Rails as future product line — do not build it yet. Architect for it.
Months 6–12	Launch Orbital Cloud publicly. First 3–5 enterprise conversations start — but sell them Cloud with a custom contract, not "Rails."
Months 12–18	After Series A: split GTM motion. Launch <code>orbital.com/rails</code> as a dedicated surface. Hire head of enterprise. Begin SOC 2 Type I.
Year 2–3	Rails becomes distinct SKU with separate pricing. SOC 2 Type II. First regulated-money flagship customer.
Year 3+	Rails overtakes Cloud in revenue. Consider spinning out the Foundation for standards governance.

The trap to avoid: do not build Rails as a separate codebase or separate company to "protect" the OSS side. Several YC companies have tried this and all have failed the same way: the enterprise code diverges from OSS core, developers notice the OSS version is getting less love, adoption stalls, and the enterprise side never closes enough deals alone to survive. *The OSS engagement is the enterprise deal's top-of-funnel.* Sever them and you amputate your own sales pipeline.

Core → Cloud: What Cloud Fixes

Core is today ~260 lines in `packages/pulse-core/src/EventEngine.ts` plus the webhook + React packages. It works — but has *structural* limitations that Cloud solves. These are not missing features; they are the architectural reasons a serious team cannot run Core in production.

Core's limitation (in current code)	Why it's a limitation	How Cloud fixes it
In-memory registry — <code>apps/server/src/registry.ts:32</code> holds <code>Map<string, WebhookRegistration></code> . Server restart = all lost.	No production fintech tolerates losing webhooks on deploy.	Managed PostgreSQL persistence. Registrations survive restarts, deployments, crashes.
Single-process SSE stream — <code>EventEngine.ts:133</code> runs one <code>server.payments().stream()</code> . Single point of failure.	No HA, no horizontal scale, no graceful failover.	Multi-region distributed ingestion with consensus on dedup. One stream dies → another picks up from same cursor.
Payment-only normalization — <code>EventEngine.ts:207</code> hard-rejects anything <code>r.type !== "payment"</code> .	Misses DEX ops, trustlines, account creation, Soroban invocations — ~70% of Stellar's actual traffic.	Full operation taxonomy + Soroban event decoding via ABI registry.
7-day Horizon history only — no replay store.	Cannot backfill new subscribers; cannot answer "what did I miss last month?"	Archival database + replay API. Full event history, queryable.
Backoff without dead-letter — <code>pulse-webhooks/src/index.ts:53</code> emits <code>webhook.failed</code> after 3 tries and moves on.	Silent data loss if consumer is down >7s.	Persistent queue with backoff over hours/days, DLQ, manual replay UI.

Core's limitation (in current code)	Why it's a limitation	How Cloud fixes it
Single API key — <code>apps/server/src/auth.ts:4</code> reads one <code>process.env.API_KEY</code> .	No multi-tenancy, team accounts, key rotation, or scoped permissions.	Multi-tenant auth, OAuth, RBAC, key scoping by address/event-type, audit log.
No observability — logs to stdout.	Cannot see delivery rates, latency percentiles, failure patterns.	Dashboards: event volume, p50/p95/p99 delivery latency, anomaly alerts.
Self-hosted ops burden	Most fintechs do not want to operate this.	99.95% managed SLA. Deploy once, forget.

One-line summary: Core = *a correct implementation*. Cloud = *a resilient, multi-tenant, historically-aware operation of that implementation*.

Cloud → Rails: What Rails Fixes

Cloud solves "I don't want to operate infrastructure." It does *not* solve "I'm a regulated entity and my compliance team won't let me integrate you." That is the Rails gap.

Cloud's limitation	Why it blocks regulated-money buyers	How Rails fixes it
Shared multi-tenant infra	A central bank or tier-1 fintech will not share an event bus with random startups.	Dedicated tenancy / VPC / customer-specific regions.
SOC 2 / PCI-DSS / ISO 27001 absent	Procurement hard-blocks. No certs = no RFP response.	Full compliance certification stack. Contract line item, not roadmap item.
Vendor trust required	Banks need to <i>prove</i> transactions to regulators, not <i>trust</i> a webhook payload from you.	ZK-attested webhooks. Each delivery carries a succinct proof. Verifiable without trusting Orbital.
No regulator-specific exports	FATF travel rule, OFAC screening, SAR reporting all need specialized formats.	SEP-24/31 lifecycle tracking, sanctions adapters, regulator-accepted audit-log exports.
Generic 99.95% SLA	Not enforceable against bank-grade availability requirements.	99.99% contractual SLA with financial penalties. Per-customer status page.
Credit-card self-serve billing	No central bank procures via Stripe checkout.	Enterprise MSA, DPA, purchase orders, annual contracts.
Generic data residency	GDPR, India RBI, Nigeria CBN, CBDC jurisdictions each have specific rules.	Sovereign deployment options (EU, India, CBDC-specific air-gapped).
No priority / latency guarantees	Market makers and compliance desks have wildly different latency needs.	Priority-fanout market — sub-100ms guaranteed for premium tier, shared for free.
No dedicated support	Bank outages at 2am do not wait for a Discord reply.	Named CSM, 24/7 on-call with SLA, dedicated solutions engineer per account.

One-line summary: Cloud = *operated infrastructure*. Rails = *operated infrastructure with cryptographic trust, regulatory certifications, and enterprise procurement surfaces*.

Repository Analysis — What Exists, What Should Be Closed

What is in the current codebase

- `packages/pulse-core` — Horizon SSE subscriber, in-memory watcher registry, payment-only normalization, exponential backoff reconnect (1s → 30s cap), EventEmitter-based pub/sub, graceful stop via `addStopHandler`.
- `packages/pulse-webhooks` — HMAC-SHA256 signing, exponential backoff retry (3 attempts default), `AbortController` timeout, timing-safe HMAC verification.
- `packages/pulse-notify` — React `useStellarEvent`, `useStellarPayment`, `useStellarActivity` hooks via `EventSource`.
- `apps/server` — Express server, in-memory `WebhookRegistry`, SSRF-safe URL validation, simple API-key auth, SSE endpoint, Stellar public-key validation, graceful shutdown.

Should any of it be closed source today?

No. Not a line of it.

Everything in the current repo is *commodity plumbing*:

- `EventEngine.ts` is a well-structured Horizon SSE client with reconnect. Any competent engineer writes this in a week.
- `WebhookDelivery` is standard HMAC + exponential backoff. Every webhook tutorial on the internet has it.
- `useStellarEvent` is 70 lines of `EventSource` wrapping — textbook React.
- `WebhookRegistry` and `routes.ts` are a reference Express implementation. Useful as a starting point, not a moat.

Closing this code would be **strictly negative**: lose OSS-driven developer acquisition (most effective GTM for dev tools), lose grant eligibility (SDF and SCF heavily favor open contributions), lose free community contributions — and gain nothing, because a competitor could rebuild this in a month.

Where the closed-source value will live (future private repo)

1. **Multi-region ingestion orchestrator** — control plane running N Horizon/RPC streams across regions, deduplicating, reaching consensus, handling failover.
2. **Persistent registry + replay store** — PostgreSQL + archival database + replay API.
3. **Multi-tenant control plane** — auth, billing, RBAC, rate limiting, customer isolation.
4. **Intent compiler** — natural-language-to-subscription-graph service.
5. **ZK attestation generator** — proof-production service turning each event into a verifiable claim.
6. **Compliance export engine** — FATF / OFAC / SAR adapters, regulator-specific formatters.
7. **Priority-fanout auction** — Soroban contract + off-chain auction matching publishers to subscribers by priority tier.
8. **Observability / analytics engine** — dashboards, anomaly detection, proprietary data layer.

These should live in a **private monorepo** that imports `@orbital/pulse-core`, `@orbital/pulse-webhooks`, `@orbital/pulse-notify` from npm exactly the way `apps/server` does today. Public packages become your stable published API surface. The private layer is where the moat is built.

The Grand-Scheme Architecture

PUBLIC REPO (MIT, npm, GitHub – the world-facing layer)

@orbital/pulse-core —
EventEngine, Watcher

@orbital/pulse-webhooks — imports core
WebhookDelivery, verifyWebhook

@orbital/pulse-notify — imports core
useStellarEvent, useStellarPayment

apps/server (reference) — imports all three (self-host example)

Purpose: developer acquisition, community trust, standards

Moat: none directly – indirectly creates brand halo

| imports as npm dependencies

PRIVATE REPO (closed – Orbital Cloud)

Control plane: multi-tenant auth, RBAC, billing, rate limits
Persistence: Postgres registry, replay store, event archive
Orchestration: multi-region ingestion, dedup, failover, HA
Observability: dashboards, analytics, anomaly detection

Purpose: developer revenue (\$29-\$499/mo SaaS)

Moat: operations quality, uptime history, scale economics

| extends Cloud with enterprise layer

PRIVATE ENTERPRISE LAYER (closed – Orbital Rails)

ZK attestation generator
Compliance exports (FATF, OFAC, SAR, SEP-24/31 lifecycle)
Dedicated tenancy / sovereign deployment
Priority-fanout auction (Soroban contract + off-chain match)
Audit logging, regulator-ready reporting
SOC 2 / PCI-DSS / ISO 27001 certified infrastructure

Purpose: enterprise revenue (\$50K-\$500K/yr contracts)

Moat: certifications + cryptographic trust + operating history

How value flows through the stack

- A developer `npm install @orbital/pulse-core` to learn Stellar events. **Free.**
- They graduate from self-hosting to Cloud because they don't want to run Postgres + HA. **\$29–\$499/mo.**
- Their company lands a fintech deal that requires SOC 2. They upgrade to Rails. **\$50K–\$500K/yr.**
- A CBDC pilot picks Rails directly, skipping Cloud. **\$500K–\$2M/yr.**

Each tier is a strict superset of the one below. No tier requires a rewrite. The same Core primitives flow all the way up — they get wrapped in progressively more infrastructure, trust, and certification.

Mental model in one line: Core is the *protocol*. Cloud is the *hosted protocol*. Rails is the *hosted + certified + cryptographically-attested protocol*. You publish the protocol, you sell the hosting, and the certification is what makes the hosting worth 100× more.

Open Questions & Next Discussion

The team's current working hypothesis (to discuss next)

"The current codebase is limited in what it can do — that is why it is in OSS, to boost its capabilities to almost what Cloud should do. The limitation for Core → Cloud then becomes *how Cloud uses the Core*, not what additional features Cloud adds."

Why this reframe is interesting

- It inverts the standard open-core playbook (where OSS is deliberately weaker than the hosted version).
- It says: *make the OSS as capable as possible*. Let it be self-hostable at production grade. The moat is then purely operational — reliability, scale, certs.
- This is closer to the Grafana / HashiCorp / Elastic model than the Supabase / Vercel / Clerk model. Both work, but for different buyer personas.

Questions this reframe surfaces (for the next session)

1. What specifically should the OSS gain to approach Cloud parity — persistence, multi-region, replay, ABI decoding, intent compiler?
2. If OSS is near-parity, how does Cloud justify price? Operating a production-grade Horizon-streaming service across regions with 99.95%+ SLA is non-trivial — but is that enough justification?
3. Does near-parity OSS cannibalize Cloud, or expand the top-of-funnel? Elastic and MongoDB both cannibalized then re-captured via license changes (SSPL, Elastic License) — a painful lesson Orbital should study now, not later.
4. Does this change the Rails strategy? Probably not — Rails moat is trust / certification / ZK / regulatory, and those are structurally out of OSS reach.
5. What would the "Orbital Licensing Decision" look like — MIT forever, or plan to migrate to source-available for later OSS components (ABI registry, intent compiler) that might compete with Cloud?

Recommended next meeting agenda

1. Agree the north-star framing: *programmable runtime vs. event infrastructure*.

2. Agree the buyer persona: *dev-tool* vs. *regulated-money infrastructure*.
3. Decide the OSS scope: *deliberately-weaker OSS* vs. *near-parity OSS*.
4. Identify the first three closed-source modules (outside the OSS repo) to start building now: *persistent control plane*, *intent compiler*, *ZK attestation generator*.
5. Draft the SEP (Stellar Ecosystem Proposal) to lead with — intent compiler, standardized event schema, or priority-fanout reactor spec.
6. Commit Week-1 engineering decisions: PostgreSQL migration, `EventEngine` refactor to extension points, v1.0.0 stability pledge.